

Stringhe

flavio

May 11, 2026

Contents

1	Le stringhe	1
1.1	ottenere la lunghezza	1
1.2	conversione a maiuscolo e minuscolo	1
1.3	ottenere i singoli caratteri	1
1.4	creazione una sottostringa	2
1.5	Concatenazione	2
1.6	Ricerca in una stringa	2
1.7	Sostituire caratteri e stringhe	2
1.8	dividere le stringhe	2
2	Confronto tra oggetti	3
2.1	differenza tra equals e ==	3

1 Le stringhe

in java nonostante le apparenze le stringhe non sono un tipo primitivo, ma sono importate dal package `java.lang` perché l'operatore `+` viene ridefinito. Inoltre dato che non un tipo primitivo non possiamo utilizzare i normali operatori che abbiamo utilizzato per accedere

1.1 ottenere la lunghezza

La classe `String` è dotata del metodo `length()` ad esempio: `String s = "Ciao"`
`System.out.println(s.length())` stamperà 4

1.2 conversione a maiuscolo e minuscolo

le stringhe hanno i metodi `toLowerCase()` e `toUpperCase()` che ritornano rispettivamente delle nuove stringhe in minuscolo o maiuscolo.

```
String min = "Ciao".toLowerCase(); // "ciao"
String max = "Ciao".toUpperCase(); // "CIAO[[id:74295e9a-da30-42fe-bb04-de5e6d5a4166]]"
String ariMin = max.toLowerCase(); // "ciao"
System.out.println("Ciao");
```

1.3 ottenere i singoli caratteri

per questo esiste il metodo `charAt`. Il primo carattere in posizione 0; l'ultimo carattere in posizione `stringa.length()-1` **importante** questo metodo ritorna molto utile per iterare sulla stringa carattere per carattere. `for(int i = 0; i < s.length(); i++) { System.out.println(s.charAt(i)); }`

1.4 creazione una sottostringa

con il metodo `s.substring(startIndex, endIndex)`. `startIndex` e `endIndex` devono essere compresi tra 0 e `s.length()-1`

- `startIndex`: indice di partenza
 - `endIndex`: indice di fine

1.5 Concatenazione

Si possono concatenare 2 stringhe:

```
String s3 = s1+s2
String s4= s1.concat(s2)
```

StringBuilder ma se si vogliono concatenare più stringhe si usa la classe **StringBuilder** che ha i metodi **append** (concatena la stringa) e **insert(int posizione, String s)**, dopo che le operazioni sulle stringhe sono state completate per ottenere la stringa si chiama il metodo `toString()` dell'istanza dello **StringBuilder** `StringBuilder sb = new StringBuilder() sb.append(s1)`

1.6 Ricerca in una stringa

Si può cercare la prima posizione di un carattere `c` con `:` restituisce -1 se il carattere, si applica alle stringhe dove ritorna l'indice del primo carattere della stringa cercata. `s1.indexOf(c)`

1.7 Sostituire caratteri e stringhe

Con il metodo `replace` è possibile sostituire tutti le occorrenze di un carattere o di una stringa con un'altra stringa.

1.8 dividere le stringhe

```
String[] s1 = "uno due tre".split(" ");  
for(int i = 0; i < s1.length; i++)  
System.out.println(s1[i]);
```

Una stringa può essere divisa in più stringhe grazie al metodo `split` se passiamo un'espressione regolare `s` e restituisce un array di sottostringhe separate da `s`.

2 Confronto tra oggetti

Tutti gli oggetti incluse le stringhe vanno confrontate con il metodo `equals`.

2.1 differenza tra `equals` e `==`

L'operatore `==` in caso di oggetti confronta il riferimento cioè l'indirizzo in memoria. Quindi è vero solo se si confrontano se puntano alla stessa cosa. Il metodo `equals` confronta la stringa carattere per carattere e restituisce `true` se le stringhe contengono la stessa sequenza di caratteri

```
String s1 = "ciao";  
String s2 = "ci"+"ao";  
String s3 = "hello";  
System.out.println(s1 == s2); //falso  
System.out.println(s1.equals(s2)); // vero  
System.out.println(s1.equals(s3)); // falso
```